

ICS 2207: Scientific Computing - Lab Week 1

Free Fall Analysis and Numerical Simulation

Student Report

1. Introduction

This notebook explores the fundamental physics of free-falling objects using Python. We investigate how different initial conditions (height and velocity) affect the motion of an object under gravity. The analysis covers analytical calculations, numerical simulations, and visualizations.

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt

# Setting a sleek style for plots
plt.style.use('bmh')
%matplotlib inline

# Global Constants
G = 9.81 # Gravitational acceleration (m/s^2)
```

2. Core Functions

We define reusable functions to calculate height and velocity based on the equations of motion:

$$h(t) = h_0 + v_0 t - \frac{1}{2} g t^2$$

$$v(t) = v_0 - g t$$

```
In [ ]: def calculate_height(t, h0, v0, g=G):
        """Calculates height at time t."""
        return h0 + v0 * t - 0.5 * g * t**2

def calculate_velocity(t, v0, g=G):
    """Calculates velocity at time t."""
    return v0 - g * t

def get_impact_time(h0, v0, g=G):
    """
    Calculates the time it takes to hit the ground (h=0).
    Uses the quadratic formula:  $-0.5*g*t^2 + v_0*t + h_0 = 0$ 
    """
    a = -0.5 * g
    b = v0
    c = h0

    #  $t = (-b \pm \sqrt{b^2 - 4ac}) / (2a)$ 
    determinant = b**2 - 4*a*c
    t1 = (-b + np.sqrt(determinant)) / (2*a)
    t2 = (-b - np.sqrt(determinant)) / (2*a)

    # We return the positive time root
    return max(t1, t2)
```

3. Practical Tasks: Analyzing Different Scenarios

We analyze three distinct cases to see how initial parameters influence the trajectory.

```
In [ ]: scenarios = [
        {"label": "Case 1: Dropped from 250m", "h0": 250, "v0": 0, "color": "#007bff"},
        {"label": "Case 2: Thrown Downward (-5m/s) from 100m", "h0": 100, "v0": -5, "color": "#dc3545"},
        {"label": "Case 3: Thrown Upward (+20m/s) from 100m", "h0": 100, "v0": 20, "color": "#28a745"}
    ]

results = []

for scenario in scenarios:
    t_impact = get_impact_time(scenario['h0'], scenario['v0'])
```

```

t_values = np.linspace(0, t_impact, 200)
h_values = calculate_height(t_values, scenario['h0'], scenario['v0'])
v_values = calculate_velocity(t_values, scenario['v0'])

results.append({
    "label": scenario['label'],
    "t": t_values,
    "h": h_values,
    "v": v_values,
    "t_impact": t_impact,
    "color": scenario['color']
})

print(f"{scenario['label']}: Impact Time = {t_impact:.4f} s")

```

4. Visualization

The following plots compare the **Height** and **Velocity** of the object across all three scenarios.

```

In [ ]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Height vs Time Plot
for res in results:
    ax1.plot(res['t'], res['h'], label=res['label'], color=res['color'], linewidth=2.5)

ax1.set_title("Height vs Time", fontsize=14, fontweight='bold')
ax1.set_xlabel("Time (s)")
ax1.set_ylabel("Height (m)")
ax1.legend()
ax1.grid(True, alpha=0.3)

# Velocity vs Time Plot
for res in results:
    ax2.plot(res['t'], res['v'], label=res['label'], color=res['color'], linewidth=2.5)

ax2.set_title("Velocity vs Time", fontsize=14, fontweight='bold')
ax2.set_xlabel("Time (s)")
ax2.set_ylabel("Velocity (m/s)")
ax2.legend()
ax2.grid(True, alpha=0.3)

```

```
ax2.axhline(0, color='black', linestyle='--', linewidth=1, alpha=0.5)

plt.tight_layout()
plt.show()
```

5. Reflection Questions

1. Why is the height-time graph curved while the velocity-time graph is straight?

This is due to the mathematical nature of the equations derived from Newton's laws:

- **Height (h)** depends on the **square of time (t^2)**, which creates a parabolic (curved) graph. This non-linearity reflects the fact that as the object falls, it covers more distance in each successive second because its speed is increasing.
- **Velocity (v)** is a **linear function of time (t)** ($v = v_0 - gt$). Since gravity (g) is constant, the rate of change of velocity (acceleration) is constant, resulting in a straight line with a slope equal to $-g$.

2. What assumptions make this model simple?

This model is idealized because it assumes:

- **No Air Resistance:** It ignores the drag force that opposes motion.
- **Constant Gravity:** It assumes g is exactly 9.81 m/s^2 through the entire fall (gravity actually weakens slightly as altitude increases).
- **No Rotation or Wind:** It ignores lateral forces or the effects of the Earth's rotation (Coriolis effect).

3. In what real situations would air resistance become important?

Air resistance becomes a critical factor when:

- **High Speeds:** The object falls from a very high altitude (like a skydiving jump).
- **Low Mass/Large Surface Area:** Objects like feathers, parachutes, or sheets of paper have a high surface-area-to-mass ratio.
- **High Fluid Density:** Falling through water or oil instead of air.

4. How does this notebook demonstrate the meaning of scientific computing?

Scientific computing is the intersection of **Physics**, **Mathematics**, and **Programming**. This notebook demonstrates it by:

- **Modeling:** Translating physical laws into mathematical equations.
- **Implementation:** Converting those equations into reusable Python functions.
- **Experimentation:** Testing "what-if" scenarios (changing heights and velocities) without manual calculation.
- **Visualization:** Turning raw numbers into intuitive graphs to uncover patterns.

6. Challenge Task

Objective: Create a function that returns the time when the object reaches a given target height.

```
In [ ]: def time_at_target_height(target_h, h0, v0, g=G):
    """
    Returns the positive time(s) when the object reaches target_h.
    Formula: target_h = h0 + v0*t - 0.5*g*t^2
    Rearranged: -0.5*g*t^2 + v0*t + (h0 - target_h) = 0
    """
    a = -0.5 * g
    b = v0
    c = h0 - target_h

    discriminant = b**2 - 4*a*c

    if discriminant < 0:
        return "Target height is never reached."

    t1 = (-b + np.sqrt(discriminant)) / (2 * a)
    t2 = (-b - np.sqrt(discriminant)) / (2 * a)

    # Filter for non-negative times and sort them
    significant_times = [round(t, 4) for t in [t1, t2] if t >= 0]

    if not significant_times:
        return "Target height was reached in the past (negative time)."

    return sorted(list(set(significant_times)))

# Testing the Challenge Function
```

```
target = 20
h_start = 100
v_start = 20

times = time_at_target_height(target, h_start, v_start)
print(f"For initial height {h_start}m and velocity {v_start}m/s,")
print(f"the object reaches {target}m height at time(s): {times} seconds.")
```

Conclusion

Through this lab, we have successfully modeled free-fall motion in Python. We observed how initial velocity can either shorten or prolong the fall (upward vs downward throw) and how quadratic equations govern the displacement of objects under constant acceleration. This foundational workflow is the basis for more complex simulations in scientific computing.